



Impacts énergétiques du projet et de ses retombées

Projet Genie Logiciel - gl16

ENSIMAG 2025/2026

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

Table des matières

1	Introduction	2
2	Moyens d'évaluation de la consommation énergétique	2
3	Mesure de l'impact énergétique : méthodologie	2
4	Efficiency du code produit : mesures avec ima -s	3
4.1	Procédure expérimentale	3
4.2	Résultats (exemples sans objet et avec objet)	3
4.3	Mesure de ressources système lors de l'exécution IMA	4
5	Analyse de l'efficacité énergétique	5
5.1	Mesure globale des tests Maven	5
5.2	Comparaison avec un script dédié et tests ciblés	5
5.3	Pistes d'optimisation énergétique (sans nuire à la qualité)	6
6	Efficiency du code produit (Deca vers IMA)	6
6.1	Choix de génération et impact énergétique	6
6.2	Optimisation du processus	7
7	Discussion et limites	7
8	Conclusion	7

1 Introduction

Tout au long du projet Génie Logiciel, nous avons été amenés à réfléchir non seulement à la correction fonctionnelle du compilateur `decac`, mais également à l'efficacité de notre produit et de notre processus de développement. Cette réflexion est directement liée à l'impact énergétique du projet, qui se manifeste à deux niveaux distincts. D'une part, le compilateur produit des programmes exécutables qui consomment de l'énergie lors de leur exécution. Les choix de génération de code ont donc un impact direct sur la consommation énergétique des logiciels produits. D'autre part, le processus de fabrication lui-même — compilation, exécution des campagnes de tests et validations répétées — constitue une source non négligeable de consommation de ressources. Ce document présente les moyens mis en œuvre pour évaluer cet impact énergétique, ainsi qu'une analyse de nos choix de conception et de validation, en s'appuyant principalement sur des mesures réalisées sur notre propre projet.

2 Moyens d'évaluation de la consommation énergétique

L'évaluation de la consommation énergétique a été réalisée de manière indirecte, en utilisant des indicateurs classiques de consommation de ressources systèmes. Nous avons principalement utilisé la commande `/usr/bin/time -v`, qui permet de mesurer :

- le temps CPU utilisateur et système ;
- le temps réel écoulé (temps mur) ;
- l'occupation mémoire maximale (RSS) ;
- les accès mémoire (page faults) ;
- les entrées/sorties sur le système de fichiers.

Ces indicateurs fournissent une approximation raisonnable de la consommation énergétique, dans la mesure où celle-ci est fortement corrélée à l'utilisation du processeur, de la mémoire et des entrées/sorties. Dans le cas de l'exécution des programmes compilés sur la machine virtuelle IMA, nous considérons, conformément aux consignes du projet, que le nombre d'instructions et de cycles exécutés constitue une approximation pertinente de la consommation énergétique d'un programme.

3 Mesure de l'impact énergétique : méthodologie

L'objectif de ce document est d'analyser l'efficacité énergétique du projet `decac`, à la fois : (i) via l'efficacité du code produit (programme IMA exécuté), et (ii) via l'efficacité du procédé de fabrication (compilation et exécution de la batterie de tests).

Conformément aux recommandations du sujet, nous utilisons des métriques indirectes de consommation énergétique :

- pour l'exécution d'un programme compilé, le processeur virtuel `ima` fournit un indicateur de coût via le nombre d'instructions exécutées et le *temps d'exécution* (un nombre de cycles sur la machine virtuelle) ;
- pour les scripts de compilation et de validation, la commande `/usr/bin/time -v` fournit des informations sur l'utilisation des ressources (temps CPU, mémoire, E/S disque et changements de contexte).

4 Efficience du code produit : mesures avec ima -s

4.1 Procédure expérimentale

Pour un programme `.deca`, nous générons d'abord le fichier assembleur `.ass` puis nous l'exécutons avec `ima` en mode statistiques :

```
./src/main/bin/decac -r 16 -d <fichier>.deca
/home/manuela/ensimag/GL/global/bin/ima -s <fichier>.ass
```

Le mode `-s` affiche notamment :

- le nombre d'instructions exécutées ;
- le temps d'exécution (approximation du nombre de cycles sur la machine virtuelle).

4.2 Résultats (exemples sans objet et avec objet)

Nous présentons ci-dessous deux exécutions représentatives, l'une sur un test *avec objet* et l'autre sur un test *sans objet*.

Test avec objet. Fichier : `src/test/deca/codegen/valid/avec_objet/axe1/retour_void.deca`.
Sortie et statistiques `ima -s` :

Appel Sub

Nombre d'instructions : 111 Temps d'execution : 400

Test sans objet. Fichier : `src/test/deca/codegen/valid/sans_objet/arithmetic/factorial.deca`.
Sortie et statistiques `ima -s` :

3628800

Nombre d'instructions : 81 Temps d'execution : 988

Discussion. Ces deux exemples illustrent que le coût d'exécution dépend fortement de la structure du programme. Le test `factorial` exécute une boucle et des multiplications (coût élevé en *temps d'exécution*), tandis que `retour_void` réalise essentiellement des appels et un affichage. Ainsi, le nombre d'instructions n'est pas le seul facteur : la nature des opérations et le nombre d'itérations impactent fortement le *temps d'exécution* (approximation des cycles) observé avec `ima`.

Exécution avec ima -s sur le fichier retour_void.ass.

```
Command being timed: "/home/manuela/ensimag/GL/global/bin/ima -s src/test/deca/codegen/valid/avec_objet/axe1/retour_void.ass"
User time (seconds): 0.00
System time (seconds): 0.00
Percent of CPU this job got: 100%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.01
Maximum resident set size (kbytes): 2816
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
```

```
Minor (reclaiming a frame) page faults: 606
Voluntary context switches: 11
Involuntary context switches: 0
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

4.3 Mesure de ressources système lors de l'exécution IMA

Pour compléter l'analyse, nous mesurons également l'empreinte en ressources lors de l'exécution de `ima` via `/usr/bin/time -v` :

```
/usr/bin/time -v /home/manuela/ensimag/GL/global/bin/ima -s <fichier>.ass
```

Sur nos exécutions, le temps CPU et la mémoire consommée par `ima` restent très faibles (ordre de la milliseconde et quelques Mo), ce qui confirme que le critère le plus pertinent pour comparer les programmes compilés est le *temps d'exécution* retourné par `ima -s` (approximation des cycles) plutôt que le coût système de l'outil `ima` lui-même.

Exécution du compilateur avec -r 16. Commande exécutée :

```
/usr/bin/time -v ./src/main/bin/decac -r 16 -d src/test/deca/codegen/valid/sans_objet
```

Résultats :

```
Command being timed: "./src/main/bin/decac -r 16 -d src/test/deca/codegen/valid/sans_
User time (seconds): 0.26
System time (seconds): 0.04
Percent of CPU this job got: 182%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.17
Maximum resident set size (kbytes): 64504
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 11309
Voluntary context switches: 2160
Involuntary context switches: 7
Swaps: 0
File system inputs: 0
File system outputs: 80
```

5 Analyse de l'efficacité énergétique

5.1 Mesure globale des tests Maven

Afin d'estimer le coût du procédé de validation, nous exécutons l'ensemble des tests Maven en instrumentant l'exécution :

```
/usr/bin/time -v mvn test
```

Cette mesure fournit notamment le temps utilisateur, le temps système, l'utilisation mémoire maximale (RSS), ainsi que les entrées/sorties disque. Dans notre cas, l'exécution complète des tests représente une part significative de la consommation de ressources du projet : elle regroupe (i) compilation Java, (ii) exécution de tests unitaires JUnit, et (iii) exécution de campagnes de tests système (scripts `.sh`).

L'analyse des données recueillies révèle un coût matériel concret :

- **Sollicitation CPU** : Le temps total de calcul s'élève à **211,42 s** (162,85 s en temps utilisateur et 48,57 s en temps système). Avec un taux d'utilisation de **228%**, le processus sature plusieurs cœurs, augmentant la consommation instantanée.
- **Empreinte mémoire** : Le *Maximum resident set size* atteint **431 584 kbytes** (environ 421 Mo), ce qui mobilise une part non négligeable de la mémoire vive pour la JVM.
- **Activité disque (I/O)** : Le rapport indique **178 640** lectures (*file system inputs*) et **37 000** écritures, illustrant une forte activité sur le support de stockage.
- **Temps réel** : L'exécution totale dure **1 :32.71**, temps durant lequel l'ensemble des composants de la machine reste en état de haute performance.

Ces chiffres démontrent que chaque exécution intégrale "à blanc" possède un coût énergétique réel. L'optimisation vers des valeurs immédiates pré-calculées et la réduction des redondances de tests constituent donc un levier direct de réduction de l'empreinte carbone du développement du compilateur.

5.2 Comparaison avec un script dédié et tests ciblés

Plutôt que de relancer systématiquement l'intégralité de la suite de tests via Maven, nous avons mesuré l'impact d'une validation ciblée sur un sous-ensemble du projet (la génération de code objet) via un script dédié :

```
/usr/bin/time -v ./src/test/script/test-all-gencode_obj.sh
```

L'objectif est de quantifier l'économie d'énergie réalisée lorsqu'on ne teste que la partie concernée par les changements en cours de développement. Les résultats sont les suivants :

- **Réduction du temps de calcul** : Le temps processeur total passe de 211,42 s à **36,6 s**. En ciblant uniquement la génération de code, on divise par près de **6 fois** l'énergie consommée par le CPU.
- **Sobriété mémoire** : L'utilisation mémoire maximale chute de 431 Mo à seulement **76 Mo**. Cette réduction drastique s'explique par l'absence de l'infrastructure lourde de Maven, limitant ainsi la charge sur la RAM.
- **Efficacité temporelle** : Le développeur obtient un retour en **17,94 s** contre plus d'une minute trente via Maven.

Cette comparaison démontre qu’une stratégie de tests ciblés est un levier majeur d’optimisation énergétique. En limitant le périmètre de validation aux seuls modules modifiés, on réduit non seulement le temps de mobilisation des ressources système, mais on minimise aussi l’usure matérielle (E/S disque divisées par 6). Cette approche permet de maintenir une haute qualité de validation tout en ayant une empreinte environnementale nettement plus faible lors des phases itératives du projet.

5.3 Pistes d’optimisation énergétique (sans nuire à la qualité)

Les mesures précédentes suggèrent plusieurs axes d’amélioration possibles :

- éviter les recompilations inutiles (ex. limiter les phases Maven déclenchées, réutiliser les artefacts quand possible) ;
- permettre des campagnes de tests plus ciblées (par passe : lex/syntax/context/codegen) lors du développement, puis exécuter la campagne complète avant rendu ;
- réduire les E/S et relances redondantes dans les scripts (mutualisation des appels à `decac`, limitation des appels multiples à `ima` si non nécessaires).
- optimiser la génération de code : intégrer le *constant folding* (pliage de constantes) pour évaluer les expressions constantes dès la compilation. Par exemple, au lieu de générer deux instructions `LOAD` suivies d’un `ADD` pour des valeurs fixes, le compilateur peut calculer directement le résultat et générer une unique instruction de chargement d’une valeur immédiate (`LOAD resultat, R1`). Cela réduit le nombre d’instructions exécutées et, par extension, la consommation énergétique du processeur.

Ces optimisations visent à diminuer le coût énergétique du procédé (temps CPU et E/S disque) tout en conservant l’objectif principal : la validation et la qualité du compilateur.

6 Efficience du code produit (Deca vers IMA)

Le compilateur `decac` génère du code assembleur IMA à partir de programmes Deca. Les choix effectués lors de cette génération ont un impact direct sur la quantité d’instructions produites et donc sur le coût énergétique à l’exécution.

6.1 Choix de génération et impact énergétique

Plusieurs décisions de compilation influencent la consommation :

- la gestion des registres et le mécanisme de *spilling* lorsque le nombre de registres disponibles est limité ;
- l’introduction de vérifications dynamiques à l’exécution (division par zéro, dépassements, déréréférencement de `null`) ;
- la génération de code pour les appels de méthodes, l’accès aux champs et la gestion des tables virtuelles dans le cas du langage orienté objet.

Par exemple, l’option `-r` permet de limiter explicitement le nombre de registres utilisables. Une valeur faible entraîne davantage de sauvegardes sur la pile, ce qui augmente le nombre d’instructions exécutées et donc la consommation énergétique. À l’inverse, une allocation plus large de registres permet de réduire les accès mémoire. De même, l’activation des vérifications à l’exécution améliore la sûreté des programmes, mais introduit des

instructions supplémentaires. Le choix entre sûreté et efficience énergétique représente un compromis explicite assumé par le compilateur.

6.2 Optimisation du processus

Nous avons envisagé plusieurs axes d’optimisation pour réduire la consommation énergétique du processus de validation :

- Limiter les recompilations inutiles en utilisant des artefacts précompilés et en réutilisant les résultats de tests.
- Mettre en place des campagnes de tests plus ciblées (par passe de validation) pour éviter de lancer des tests non pertinents à chaque cycle de validation.
- Réduire les appels redondants et les I/O dans les scripts de validation pour diminuer le temps d’exécution global.

Ces optimisations permettent de réduire la consommation énergétique du processus sans compromettre la qualité du compilateur.

7 Discussion et limites

Les mesures présentées reposent sur des indicateurs indirects de consommation énergétique. Elles restent néanmoins représentatives des tendances observées au cours du projet. Certaines limitations identifiées lors des tests (par exemple les littéraux flottants de taille extrême ou des extensions non implémentées) sont connues et documentées. Elles n’affectent pas la validité globale de l’analyse énergétique, mais rappellent que l’optimisation doit rester compatible avec les objectifs de correction et de robustesse.

8 Conclusion

Cette étude met en évidence que les choix de compilation et de validation ont un impact réel sur la consommation énergétique du projet **decac**. L’efficience du code produit dépend directement des stratégies de génération adoptées, tandis que l’efficience du procédé de fabrication repose sur l’organisation et l’exécution raisonnée des campagnes de tests. La prise en compte de ces aspects tout au long du projet permet d’atteindre un compromis équilibré entre qualité du compilateur, robustesse et maîtrise de la consommation de ressources.